Instituto Tecnológico Superior del Oriente del Estado de Hidalgo

Título: Reporte Ejecutivo: Métodos Numéricos de Derivación e Integración

Materia: Métodos numéricos

Institución: ITESA

Autor(es): Emanuel Islas Quintos

Profesor: Efren Rolando Romero León

Fecha: 24/ 04/2026

Grupo / Semestre: EJ26

1. INTRODUCCIÓN 4

2. DESARROLLO 4

2.1 MÉTODOS DE DERIVACIÓN 4

2.1.1 MÉTODO DE TRES PUNTOS 4

Descripción del método 4

Fórmula matemática 5

Algoritmo paso a paso 5

2.1.2 MÉTODO DE CINCO PUNTOS 5

Descripción del método 5

Fórmula matemática 5

Algoritmo paso a paso 6

2.2 MÉTODOS DE INTEGRACIÓN 6

2.2.1 MÉTODO DEL TRAPECIO 6

Descripción del método 6

Fórmula matemática 6

Algoritmo paso a paso 7

2.2.2 REGLA DE SIMPSON 7

Descripción del método 7

Fórmula matemática 7

Algoritmo paso a paso 8

2.2.3 CUADRATURA GAUSSIANA 8

Descripción del método 8

Fórmula matemática 8

Algoritmo paso a paso 9

3. EJERCICIOS 9

3.1 EJERCICIOS: MÉTODO DE TRES PUNTOS 9

Ejercicio 1 (Caso Ideal) 9

Salida 10

CONCLUSIÓN 11

Ejercicio 2 (Caso con Error) 11

Salida 12

CAUSA 13

SOLUCIÓN 13

Ejercicio 3 (Caso Extra) 13

Salida 14

CONCLUSIÓN 15

3.2 EJERCICIOS: MÉTODO DE CINCO PUNTOS 15

Ejercicio 1 (Caso Ideal) 15

Salida 16

Ejercicio 2 (Caso con Error) 17

Salida 18

ANÁLISIS 18

CAUSA 19

SOLUCIÓN 19

Ejercicio 3 (Caso Extra) 19

Salida 20

CONCLUSIÓN 21

3.3 EJERCICIOS: MÉTODO DEL TRAPECIO 21

Ejercicio 1 (Caso Ideal) 21

Salida 22
CONCLUSIÓN 23
Ejercicio 2 (Caso con Error) 23
Salida 24
CAUSA 24
REGLA PRÁCTICA 24
SOLUCIÓN 24
Ejercicio 3 (Caso Extra) 25
Salida 26
RESULTADOS 26

3.4 EJERCICIOS: REGLA DE SIMPSON 27
Ejercicio 1 (Caso Ideal) 27
Salida 28
CONCLUSIÓN 28
Ejercicio 2 (Caso con Error) 28
Salida 29
ERROR 30
CAUSA 30
NOTA 30
Ejercicio 3 (Caso Extra) 30
Salida 32

3.5 EJERCICIOS: CUADRATURA GAUSSIANA 32
Ejercicio 1 (Caso Ideal) 32
Salida 33
CONCLUSIÓN 34
Ejercicio 2 (Caso con Error) 34
Salida 35
CAUSA 36
SOLUCIÓN 36
Ejercicio 3 (Caso Extra) 36
Salida 37
CONCLUSIÓN 37

Commit a GitHub 38

1. INTRODUCCIÓN

En ingeniería, física, economía y prácticamente cualquier disciplina científica, es frecuente necesitar calcular derivadas e integrales de funciones que no siempre tienen una forma analítica sencilla o, incluso, ni siquiera se conocen de forma explícita. En esos casos, los métodos numéricos ofrecen una alternativa poderosa y práctica: permiten aproximar con suficiente precisión tanto derivadas como integrales a partir de datos discretos o de evaluaciones puntuales de la función.

Este reporte estudia cinco métodos numéricos fundamentales, organizados en dos grandes bloques. En el bloque de derivación se presentan el método de tres puntos y el método de cinco puntos, ambos basados en diferencias finitas. En el bloque de integración se analizan el método del trapecio, la regla de Simpson y la cuadratura gaussiana, cada uno con filosofías y rangos de aplicación distintos.

Para cada método se expone su fundamento teórico, la fórmula matemática con explicación de variables, el algoritmo paso a paso y tres ejercicios resueltos en Python: un caso ideal (donde el método produce resultados precisos), un caso con error (donde el método falla o es inadecuado) y un caso extra (más complejo o aplicado a un contexto diferente). Al final del documento se incluye el código Python completo, organizado por clases, que ejecuta todos los casos automáticamente.

2. DESARROLLO

2.1 MÉTODOS DE DERIVACIÓN

2.1.1 MÉTODO DE TRES PUNTOS

Descripción del método

El método de tres puntos es una fórmula de diferencias finitas que aproxima la derivada de una función en un punto usando tres valores conocidos de esa función. Es una extensión de las simples diferencias hacia adelante o hacia atrás, y su principal ventaja es que logra una precisión de segundo orden (error proporcional a h^2), lo cual lo hace bastante más exacto que las diferencias simples de primer orden.

Existen dos versiones principales. La fórmula del extremo izquierdo usa los tres puntos $f(x_0)$, $f(x_0+h)$ y $f(x_0+2h)$ y es útil cuando solo se tienen datos hacia adelante. La fórmula del punto medio usa $f(x_0-h)$ y $f(x_0+h)$, es decir, un punto a cada lado, y entrega la mejor precisión posible con tres puntos.

Se usa cuando se dispone de datos experimentales con espaciado uniforme y se necesita una estimación razonablemente precisa de la derivada sin incurrir en un costo computacional alto.

Ventajas: fácil de implementar, buena precisión para funciones suaves, error de orden $O(h^2)$. Desventajas: sensible a ruido en los datos, no es adecuado cuando h es muy grande o la función tiene cambios abruptos.

Fórmula matemática

Fórmula del punto medio (la más precisa con tres puntos):

$f'(x_0) \approx [\ f(x_0 + h) - f(x_0 - h)\ ] / (2h)$

Fórmula del extremo izquierdo (cuando no hay punto a la izquierda):

$f'(x_0) \approx [\ -3 \cdot f(x_0) + 4 \cdot f(x_0 + h) - f(x_0 + 2h)\ ] / (2h)$

Fórmula del extremo derecho (cuando no hay punto a la derecha):

$f'(x_0) \approx [\ f(x_0 - 2h) - 4 \cdot f(x_0 - h) + 3 \cdot f(x_0)\ ] / (2h)$

Donde:

- $f(x)$ es la función cuya derivada se desea aproximar
- x_0 es el punto donde se evalúa la derivada
- h es el paso o espaciado entre puntos (debe ser positivo y pequeño)
- $f'(x_0)$ es la aproximación de la derivada en x_0

Algoritmo paso a paso

1. Definir la función $f(x)$ o los datos tabulados.
2. Elegir el punto x_0 donde se quiere calcular la derivada.
3. Definir el paso h (debe ser pequeño; típicamente entre 0.001 y 0.1 dependiendo del problema).
4. Evaluar la función en los puntos necesarios: $f(x_0 - h)$, $f(x_0)$, $f(x_0 + h)$, etc.
5. Sustituir en la fórmula correspondiente (punto medio, extremo izquierdo o derecho).
6. Calcular y reportar el resultado. Si se conoce la derivada exacta, calcular el error relativo.

2.1.2 MÉTODO DE CINCO PUNTOS

Descripción del método

El método de cinco puntos extiende la idea del método de tres puntos usando cinco evaluaciones de la función, lo que eleva la precisión al orden $O(h^4)$. Esto significa que si h se reduce a la mitad, el error disminuye 16 veces, en lugar de solo 4 veces como ocurre con el método de tres puntos.

Es especialmente útil cuando se necesita alta precisión en la derivada y la función es suave (sin discontinuidades ni cambios bruscos). En análisis estructural, simulaciones físicas y procesamiento de señales de alta fidelidad, este método es preferido sobre el de tres puntos.

Su desventaja es que requiere cinco puntos igualmente espaciados alrededor del punto de interés, lo que puede ser una limitación si los datos no están disponibles en ese rango, o si la función tiene comportamientos irregulares.

Fórmula matemática

Fórmula del punto medio (la más común):

$f'(x_0) \approx [\ f(x_0 - 2h) - 8 \cdot f(x_0 - h) + 8 \cdot f(x_0 + h) - f(x_0 + 2h)\ ] / (12h)$

Donde:

- $f(x_0 - 2h)$, $f(x_0 - h)$, $f(x_0 + h)$, $f(x_0 + 2h)$ son los cuatro puntos usados (el punto central x_0 no aparece en esta fórmula específica)
- h es el paso uniforme entre puntos
- $f'(x_0)$ es la aproximación de la derivada en x_0 con error $O(h^4)$

Algoritmo paso a paso

1. Definir la función f(x).
2. Elegir el punto x_0 donde se desea estimar la derivada.
3. Seleccionar el paso h (puede ser mayor que en el método de tres puntos gracias a la mayor precisión).
4. Evaluar la función en los cinco puntos: $x_0 - 2h$, $x_0 - h$, x_0, $x_0 + h$, $x_0 + 2h$.
5. Sustituir los cuatro valores externos en la fórmula del punto medio.
6. Dividir entre 12h.
7. Reportar el resultado y comparar con la derivada exacta si se conoce.
8.

2.2 MÉTODOS DE INTEGRACIÓN

2.2.1 MÉTODO DEL TRAPECIO

Descripción del método

El método del trapecio aproxima el área bajo una curva reemplazando la función por segmentos de línea recta entre puntos consecutivos. Cada par de puntos forma un trapecio, y la suma de las áreas de todos esos trapecios es la aproximación de la integral.

Es el método de integración numérica más elemental, pero aun así tiene aplicaciones prácticas importantes: es robusto, fácil de entender y funciona bien con datos tabulados irregulares. Su error es de orden $O(h^2)$, lo que significa que duplicar el número de subintervalos reduce el error a la cuarta parte.

La versión compuesta (múltiples subintervalos) es mucho más precisa que la versión simple (solo dos puntos), y es la que se usa en la práctica.

Ventajas: simplicidad, aplicable a datos experimentales no necesariamente generados por una fórmula, bajo costo computacional. Desventajas: poco preciso con funciones muy curvas y pasos h grandes; su error es mayor que el de Simpson o la cuadratura gaussiana.

Fórmula matemática

Regla del trapecio compuesta:

$\int[a\ a\ b]\ f(x)\ dx \approx (h/2) \cdot [\ f(x_0) + 2 \cdot f(x_1) + 2 \cdot f(x_2) + \ldots + 2 \cdot f(x_{n-1}) + f(x_n)\ ]$

Donde:

- a y b son los límites de integración
- n es el número de subintervalos
- h = (b - a) / n es el ancho de cada subintervalo
- $x_0 = a$, $x_1 = a + h$, ..., $x_n = b$ son los puntos de evaluación
- $f(x_i)$ es el valor de la función en cada punto

Algoritmo paso a paso

1. Definir la función f(x) y los límites a, b.
2. Elegir el número de subintervalos n (entre más grande, más preciso).
3. Calcular el paso h = (b - a) / n.
4. Generar los puntos $x_i = a + i \cdot h$ para i = 0, 1, ..., n.
5. Evaluar $f(x_i)$ en cada punto.
6. Aplicar la fórmula: sumar el primer y último valor, más dos veces los valores intermedios, y multiplicar todo por h/2.
7. Reportar el resultado.

2.2.2 REGLA DE SIMPSON

Descripción del método

La regla de Simpson es una mejora directa sobre el método del trapecio. En lugar de aproximar la función con líneas rectas, usa polinomios de segundo grado (parábolas) que se ajustan a grupos de tres puntos consecutivos. Esta curvatura adicional captura mejor el comportamiento real de la función y reduce el error a $O(h^4)$.

La versión más común es la Regla de Simpson 1/3, que requiere un número par de subintervalos. Existe también la Regla de Simpson 3/8, que trabaja con múltiplos de tres subintervalos y es útil cuando el número de puntos no es par.

Simpson 1/3 es una de las reglas de integración más utilizadas en ingeniería y ciencias porque ofrece un balance excelente entre precisión y costo computacional.

Ventajas: mucho más precisa que el trapecio, fácil de implementar, ampliamente soportada en bibliotecas numéricas. Desventajas: requiere número par de subintervalos (versión 1/3), no funciona bien con funciones no suaves o con discontinuidades en el intervalo.

Fórmula matemática

Regla de Simpson 1/3 compuesta:

$$\int[a\ a\ b]\ f(x)\,dx \approx (h/3) \cdot [\ f(x_0) + 4 \cdot f(x_1) + 2 \cdot f(x_2) + 4 \cdot f(x_3) + 2 \cdot f(x_4) + \ldots + 4 \cdot f(x_{n-1}) + f(x_n)\ ]$$

El patrón de coeficientes es: 1, 4, 2, 4, 2, ..., 4, 1

Donde:

- n debe ser un número par
- h = (b - a) / n
- Los coeficientes alternan entre 4 y 2 para los puntos interiores

Algoritmo paso a paso

1. Definir la función f(x) y los límites a, b.
2. Elegir n par (recomendado: múltiplo de 2, por ejemplo n = 4, 6, 8, ...).
3. Calcular h = (b - a) / n.
4. Generar los n + 1 puntos: $x_i = a + i \cdot h$.
5. Evaluar $f(x_i)$ en todos los puntos.
6. Sumar: $f(x_0) + f(x_n) + 4 \times$ (suma de f en índices impares) + 2 × (suma de f en índices pares interiores).
7. Multiplicar el total por h/3.
8. Reportar el resultado.

2.2.3 CUADRATURA GAUSSIANA

Descripción del método

La cuadratura gaussiana es un método de integración que elige de manera óptima tanto los puntos de evaluación como sus pesos, de modo que se logra la mayor precisión posible con el menor número de evaluaciones. Con n puntos, puede integrar exactamente polinomios de grado hasta 2n - 1, lo cual es notable comparado con el trapecio o Simpson, que requieren muchos más puntos para lograr una precisión similar.

La idea central es transformar el intervalo [a, b] al intervalo estándar [-1, 1] y usar los nodos y pesos precalculados de los polinomios de Legendre. Estos nodos no están igualmente espaciados, sino ubicados en posiciones que minimizan el error de integración.

Es el método preferido cuando se necesita alta precisión con pocas evaluaciones, especialmente en problemas donde calcular f(x) es costoso computacionalmente (simulaciones físicas, integrales en múltiples dimensiones, etc.).

Ventajas: altísima precisión con pocas evaluaciones, excelente para funciones suaves, base de muchos métodos modernos de cuadratura adaptativa. Desventajas: requiere transformar el intervalo, los nodos no son igualmente espaciados (no sirve directamente para datos tabulados uniformes), la función debe ser suave en todo el intervalo.

Fórmula matemática

Cambio de variable del intervalo [a, b] al intervalo [-1, 1]:

$x = ((b - a) \cdot t + (a + b)) / 2$, con $t \in [-1, 1]$

$dx = (b - a) / 2 \cdot dt$

Fórmula de cuadratura gaussiana con n puntos:

$\int[a\ a\ b]\ f(x)\ dx \approx ((b - a) / 2) \cdot \Sigma_i\ w_i \cdot f(\ ((b-a)\cdot t_i + (a+b))\ /\ 2\ )$

Donde:

- t_i son los nodos gaussianos en [-1, 1] (raíces del polinomio de Legendre de grado n)
- w_i son los pesos asociados a cada nodo
- (b - a) / 2 es el factor de escala del cambio de variable
- n es el número de puntos de cuadratura (mayor n = mayor precisión)

Algoritmo paso a paso

1. Definir la función f(x) y los límites a, b.
2. Elegir el número de puntos n (típicamente entre 2 y 10 para alta precisión).
3. Obtener los nodos t_i y pesos w_i del polinomio de Legendre de grado n (precalculados o usando numpy).
4. Para cada nodo t_i, calcular el punto correspondiente en [a, b]: $x_i = ((b-a)\cdot t_i + (a+b))\ /\ 2$.
5. Evaluar $f(x_i)$ en cada punto transformado.
6. Calcular la suma ponderada: $\Sigma\ w_i \cdot f(x_i)$.
7. Multiplicar por el factor (b - a) / 2.
8. Reportar el resultado.

3. EJERCICIOS

3.1 EJERCICIOS: MÉTODO DE TRES PUNTOS

Ejercicio 1 (Caso Ideal)

Enunciado: En un experimento de transferencia de calor, la temperatura de una varilla de acero en función de la posición x (en metros) está modelada por $T(x) = e^{(-x^2)} \cdot \cos(x)$. Se desea conocer la tasa de cambio de temperatura en x = 1.0 m para ajustar el sistema de enfriamiento.

Datos: $T(x) = e^{(-x^2)} \cdot \cos(x)$, $x_0 = 1.0$, $h = 0.01$

Qué se pide: Aproximar T'(1.0) usando el método de tres puntos (fórmula del punto medio). Comparar con el valor exacto y calcular el error relativo.

```
def caso_ideal(self):
        """
        CASO IDEAL: Derivada de T(x) = e^(-x^2) * cos(x) en x = 1.0
        ------------------------------------------------------------
        Contexto: Distribución de temperatura en varilla de acero.
        Se necesita la tasa de cambio de temperatura en x=1.0 m para
        ajustar el sistema de enfriamiento.

        Datos:
```

```
            T(x) = e^(-x²) · cos(x)
            x₀ = 1.0 m
            h = 0.01

        Se pide: T'(1.0) con el método de 3 puntos (punto medio).
        Comparar con el valor exacto y calcular el error relativo.
        """
        print("\n" + "="*60)
        print("MÉTODO DE TRES PUNTOS - CASO IDEAL")
        print("Función: T(x) = e^(-x²) · cos(x)")
        print("Punto: x₀ = 1.0 | Paso: h = 0.01")
        print("Contexto: Distribución de temperatura en varilla de acero")
        print("="*60)

        def T(x):
            return math.exp(-x**2) * math.cos(x)

        def T_prima_exacta(x):
            # Regla del producto + cadena:
            # d/dx[e^(-x²)·cos(x)] = -2x·e^(-x²)·cos(x) - e^(-x²)·sin(x)
            return -2 * x * math.exp(-x**2) * math.cos(x) - math.exp(-x**2) *
math.sin(x)

        x0 = 1.0
        h = 0.01

        aprox = self.derivada_punto_medio(T, x0, h)
        exacta = T_prima_exacta(x0)
        error = abs((aprox - exacta) / exacta) * 100

        print(f"\n  Valor aproximado (3 puntos) : {aprox:.10f}")
        print(f"  Valor exacto (analítico)    : {exacta:.10f}")
        print(f"  Error relativo              : {error:.8f}%")
        print("\n  CONCLUSIÓN: Con función suave y h pequeño, el método de")
        print("  3 puntos converge con error de orden O(h²). Funciona
correctamente.")
```

Salida

MÉTODO DE TRES PUNTOS - CASO IDEAL

Función: T(x) = e^(-x²) · cos(x)

Punto: x0 = 1.0 | Paso: h = 0.01

Contexto: Distribución de temperatura en varilla de acero

==========

Valor aproximado (3 puntos) : -0.7070847629

Valor exacto (analítico) : -0.7070920963

Error relativo : 0.00103712%

CONCLUSIÓN: Con función suave y h pequeño, el método de

3 puntos converge con error de orden $O(h^2)$. Funciona correctamente.

Ejercicio 2 (Caso con Error)

Enunciado: Un técnico intenta usar el método de tres puntos con h = 0.5 para estimar la derivada de la función f(x) = tan(x) en x = π/2 - 0.01, que es un punto muy cercano a la discontinuidad de la tangente en π/2.

Datos: f(x) = tan(x), x_0 = π/2 - 0.01, h = 0.5

Qué se pide: Aplicar el método y observar cómo el paso h demasiado grande cruza la discontinuidad, produciendo un resultado completamente erróneo. El programa debe detectar y reportar la situación.

```
def caso_error(self):
        """
        CASO CON ERROR: f(x) = tan(x) cerca de la discontinuidad en π/2
        ---------------------------------------------------------------
        Problema: El paso h = 0.5 hace que f(x0+h) cruce la discontinuidad
        de tan(x) en π/2, produciendo un resultado completamente erróneo.

        Datos:
            f(x) = tan(x)
            x₀ = π/2 - 0.01 (muy cerca de la discontinuidad)
            h = 0.5 (demasiado grande, cruza π/2)

        Se pide: Observar el resultado incorrecto y entender la causa.
        """
        print("\n" + "="*60)
        print("MÉTODO DE TRES PUNTOS - CASO CON ERROR")
        print("Función: f(x) = tan(x) cerca de x = π/2")
        print("Problema: h = 0.5 cruza la discontinuidad en π/2")
        print("="*60)

        def f(x):
            return math.tan(x)

        x0 = math.pi / 2 - 0.01  # Muy cercano a la discontinuidad
        h = 0.5

        punto_derecho = x0 + h
        punto_izquierdo = x0 - h
```

```
        print(f"\n  x₀                = {x0:.6f} rad")
        print(f"  x₀ + h           = {punto_derecho:.6f} rad")
        print(f"  π/2              = {math.pi/2:.6f} rad")
        print(f"  Distancia a π/2 = {abs(punto_derecho - math.pi/2):.6f} rad")
        print("\n  ADVERTENCIA: x₀+h cruza la discontinuidad de tan(x) en π/2.")

        try:
            aprox = self.derivada_punto_medio(f, x0, h)
            exacta = 1 / math.cos(x0)**2  # Derivada de tan(x) = sec²(x)
            error_abs = abs(aprox - exacta)
            print(f"\n  Resultado incorrecto   : {aprox:.4f}")
            print(f"  Valor exacto en x₀     : {exacta:.4f}")
            print(f"  Error absoluto         : {error_abs:.2f}  ← INACEPTABLE")
            print("\n  CAUSA: f(x₀+h) evalúa tan(x) más allá de π/2, donde la
función")
            print("  cambia de signo abruptamente (de +∞ a -∞).")
            print("  SOLUCIÓN: Usar h << distancia_a_discontinuidad")
            print(f"  En este caso: h debe ser mucho menor que {abs(x0 -
math.pi/2):.4f}")

            # Mostrar corrección con h pequeño
            h_correcto = 0.0001
            aprox_correcto = self.derivada_punto_medio(f, x0, h_correcto)
            error_correcto = abs((aprox_correcto - exacta) / exacta) * 100
            print(f"\n  Con h = {h_correcto}: resultado = {aprox_correcto:.6f} |
error = {error_correcto:.6f}%  ← CORRECTO")

        except (ValueError, ZeroDivisionError, OverflowError) as e:
            print(f"\n  ERROR NUMÉRICO: {e}")
            print("  El método falla porque la función no es continua en el
intervalo evaluado.")
```

Salida

MÉTODO DE TRES PUNTOS - CASO CON ERROR

Función: f(x) = tan(x) cerca de x = π/2

Problema: h = 0.5 cruza la discontinuidad en π/2

==========

x0 = 1.560796 rad

x0 + h = 2.060796 rad

π/2 = 1.570796 rad

Distancia a π/2 = 0.490000 rad

ADVERTENCIA: x_0+h cruza la discontinuidad de tan(x) en $\pi/2$.

Resultado incorrecto : -3.6626

Valor exacto en x_0 : 10000.3333

Error absoluto : 10004.00 ← INACEPTABLE

CAUSA: $f(x_0+h)$ evalúa tan(x) más allá de $\pi/2$, donde la función cambia de signo abruptamente (de $+\infty$ a $-\infty$).

SOLUCIÓN: Usar h << distancia_a_discontinuidad

En este caso: h debe ser mucho menor que 0.0100

Con h = 0.0001: resultado = 10001.333440 | error = 0.010001% ← CORRECTO

Ejercicio 3 (Caso Extra)

Enunciado: En un sistema de control, se necesita estimar la aceleración (segunda derivada de posición) de un vehículo a partir de su función de posición $s(t) = 3\cdot t^3 - 2\cdot t^2 + t + 5$ (en metros, con t en segundos). Usando el método de tres puntos, calcular la primera y segunda derivada en t = 2.0 s.

Datos: $s(t) = 3t^3 - 2t^2 + t + 5$, $t_0 = 2.0$, $h = 0.01$

Qué se pide: Calcular s'(2.0) (velocidad) y s''(2.0) (aceleración) numéricamente y comparar ambos con los valores exactos.

```
def caso_extra(self):
        """
        CASO EXTRA: Velocidad y aceleración de vehículo
        ----------------------------------------------------------------
        s(t) = 3t³ - 2t² + t + 5 (posición en metros)
        Se calcula la primera derivada (velocidad) y la segunda (aceleración)
        usando diferencias finitas centradas de 3 puntos.

        Datos:
            s(t) = 3t³ - 2t² + t + 5
            t₀ = 2.0 s
            h = 0.01 s

        Se pide: s'(2.0) [velocidad] y s''(2.0) [aceleración], comparar con
exactas.
```

```
        """
        print("\n" + "="*60)
        print("MÉTODO DE TRES PUNTOS - CASO EXTRA")
        print("Función: s(t) = 3t³ - 2t² + t + 5 (posición en metros)")
        print("Punto: t₀ = 2.0 s | Paso: h = 0.01 s")
        print("Se calculan: velocidad s'(t₀) y aceleración s''(t₀)")
        print("="*60)

        def s(t):
            return 3*t**3 - 2*t**2 + t + 5

        def s_prima_exacta(t):
            # ds/dt = 9t² - 4t + 1
            return 9*t**2 - 4*t + 1

        def s_doble_prima_exacta(t):
            # d²s/dt² = 18t - 4
            return 18*t - 4

        t0 = 2.0
        h = 0.01

        # Primera derivada (velocidad) con fórmula de punto medio
        vel_aprox = self.derivada_punto_medio(s, t0, h)
        vel_exacta = s_prima_exacta(t0)
        error_vel = abs((vel_aprox - vel_exacta) / vel_exacta) * 100

        # Segunda derivada (aceleración) con diferencia centrada de 3 puntos:
        # s''(t0) ≈ [s(t0-h) - 2·s(t0) + s(t0+h)] / h²
        acel_aprox = (s(t0 - h) - 2*s(t0) + s(t0 + h)) / h**2
        acel_exacta = s_doble_prima_exacta(t0)
        error_acel = abs((acel_aprox - acel_exacta) / acel_exacta) * 100

        print(f"\n  VELOCIDAD en t = {t0} s:")
        print(f"    Aproximada (3 puntos) : {vel_aprox:.8f} m/s")
        print(f"    Exacta (analítica)    : {vel_exacta:.8f} m/s")
        print(f"    Error relativo        : {error_vel:.8f}%")

        print(f"\n  ACELERACIÓN en t = {t0} s:")
        print(f"    Aproximada (3 puntos) : {acel_aprox:.8f} m/s²")
        print(f"    Exacta (analítica)    : {acel_exacta:.8f} m/s²")
        print(f"    Error relativo        : {error_acel:.8f}%")

        print("\n  CONCLUSIÓN: Las diferencias finitas centradas permiten obtener
tanto")
        print("  la 1ra como la 2da derivada con alta precisión usando 3
puntos.")
```

Salida:

MÉTODO DE TRES PUNTOS - CASO EXTRA

Función: $s(t) = 3t^3 - 2t^2 + t + 5$ (posición en metros)

Punto: $t_0 = 2.0$ s | Paso: $h = 0.01$ s

Se calculan: velocidad $s'(t_0)$ y aceleración $s''(t_0)$

==========

VELOCIDAD en t = 2.0 s:

Aproximada (3 puntos) : 29.00030000 m/s

Exacta (analítica) : 29.00000000 m/s

Error relativo : 0.00103448%

ACELERACIÓN en t = 2.0 s:

Aproximada (3 puntos) : 32.00000000 m/s²

Exacta (analítica) : 32.00000000 m/s²

Error relativo : 0.00000000%

CONCLUSIÓN: Las diferencias finitas centradas permiten obtener tanto la 1ra como la 2da derivada con alta precisión usando 3 puntos.

3.2 EJERCICIOS: MÉTODO DE CINCO PUNTOS

Ejercicio 1 (Caso Ideal)

Enunciado: En aerodinámica, el coeficiente de sustentación de un perfil alar depende del ángulo de ataque θ. La función de sustentación aproximada es $L(\theta) = \sin(\theta)\cdot\cos(\theta)^2$. Se requiere la tasa de cambio de sustentación en $\theta = 0.5$ rad con alta precisión para el diseño del estabilizador.

Datos: $L(\theta) = \sin(\theta)\cdot\cos^2(\theta)$, $\theta_0 = 0.5$ rad, $h = 0.1$

Qué se pide: Calcular $L'(0.5)$ con el método de cinco puntos y con el de tres puntos. Comparar ambas precisiones con el valor exacto.

```
def caso_ideal(self):
        """
        CASO IDEAL: L(θ) = sin(θ)·cos²(θ) en θ = 0.5 rad
        ------------------------------------------------------------
        Contexto: Coeficiente de sustentación de perfil alar en función
        del ángulo de ataque θ. Se compara la precisión de 3 puntos vs 5 puntos.
```

```
        Datos:
            L(θ) = sin(θ) · cos²(θ)
            θ₀ = 0.5 rad
            h = 0.1

        Se pide: L'(0.5) con 3 y 5 puntos. Comparar errores.
        """
        print("\n" + "="*60)
        print("MÉTODO DE CINCO PUNTOS - CASO IDEAL")
        print("Función: L(θ) = sin(θ)·cos²(θ) (sustentación aerodinámica)")
        print("Punto: θ₀ = 0.5 rad | Paso: h = 0.1")
        print("="*60)

        def L(theta):
            return math.sin(theta) * math.cos(theta)**2

        def L_prima_exacta(theta):
            # d/dθ [sin·cos²] = cos³ - 2·sin²·cos
            return math.cos(theta)**3 - 2 * math.sin(theta)**2 * math.cos(theta)

        theta0 = 0.5
        h = 0.1

        # Tres puntos (diferencia centrada simple)
        aprox_3p = (L(theta0 + h) - L(theta0 - h)) / (2 * h)

        # Cinco puntos
        aprox_5p = self.derivada_primera(L, theta0, h)

        exacta = L_prima_exacta(theta0)
        error_3p = abs((aprox_3p - exacta) / exacta) * 100
        error_5p = abs((aprox_5p - exacta) / exacta) * 100

        print(f"\n  Valor exacto (analítico)   : {exacta:.12f}")
        print(f"  Aprox. 3 puntos            : {aprox_3p:.12f}")
        print(f"  Error 3 puntos             : {error_3p:.8f}%")
        print(f"  Aprox. 5 puntos            : {aprox_5p:.12f}")
        print(f"  Error 5 puntos             : {error_5p:.8f}%")

        if error_5p > 0:
            mejora = error_3p / error_5p
            print(f"\n  El método de 5 puntos es {mejora:.0f}x más preciso que el
de 3 puntos.")
        print("  Esto se debe a que el error pasa de O(h²) a O(h⁴).")
```

Salida: MÉTODO DE CINCO PUNTOS - CASO IDEAL

Función: $L(\theta) = \sin(\theta)\cdot\cos^2(\theta)$ (sustentación aerodinámica)

Punto: $\theta_0 = 0.5$ rad | Paso: $h = 0.1$

==========

Valor exacto (analítico) : 0.272448541723

Aprox. 3 puntos : 0.271290844997

Error 3 puntos : 0.42492308%

Aprox. 5 puntos : 0.272433639663

Error 5 puntos : 0.00546968%

El método de 5 puntos es 78x más preciso que el de 3 puntos.

Esto se debe a que el error pasa de $O(h^2)$ a $O(h^4)$.

Ejercicio 2 (Caso con Error)

Enunciado: Se intenta aplicar el método de cinco puntos a la función $f(x) = |x|$ en $x = 0$, que tiene una esquina (derivada no definida en sentido clásico) en ese punto.

Datos: $f(x) = |x|$, $x_0 = 0$, $h = 0.1$

Qué se pide: Aplicar el método y observar que el resultado no converge al valor real (la derivada no existe en $x = 0$). El programa debe intentar calcularlo y reportar la discrepancia, además de explicar por qué falla.

```
def caso_error(self):
        """
        CASO CON ERROR: f(x) = |x| en x = 0 (esquina, no diferenciable)
        ---------------------------------------------------------------
        El método de 5 puntos devuelve un resultado numérico (≈ 0), pero
        este valor es matemáticamente inválido porque la derivada de |x|
        no existe en x = 0 (límites laterales son +1 y -1, distintos).

        Datos:
            f(x) = |x|
            x₀ = 0
            h = 0.1

        Se pide: Aplicar el método y analizar el resultado incorrecto.
        """
        print("\n" + "="*60)
        print("MÉTODO DE CINCO PUNTOS - CASO CON ERROR")
        print("Función: f(x) = |x| en x = 0 (función con esquina)")
        print("La derivada clásica NO existe en x = 0")
        print("="*60)
```

```
        def f(x):
            return abs(x)

        x0 = 0.0
        h = 0.1

        aprox = self.derivada_primera(f, x0, h)

        print(f"\n  Resultado numérico en x = 0 : {aprox:.10f}")
        print(f"  Derivada lateral izquierda  : -1.0")
        print(f"  Derivada lateral derecha    :  1.0")
        print(f"\n  ANÁLISIS:")
        print("  La fórmula de 5 puntos usa simetría: los términos negativos y
positivos")
        print("  se cancelan y da exactamente 0.0. Esto parece 'limpio', pero es
incorrecto.")
        print("  La derivada de |x| no existe en x=0 porque los límites laterales
son distintos.")
        print("\n  CAUSA: La fórmula asume que la función es diferenciable en el
punto.")
        print("  En una esquina (cambio de pendiente abrupto), el supuesto
falla.")
        print("  SOLUCIÓN: Verificar diferenciabilidad antes de aplicar cualquier
fórmula.")
        print("            Usar derivadas laterales si la función tiene
esquinas.")
```

Salida: MÉTODO DE CINCO PUNTOS - CASO CON ERROR

Función: f(x) = |x| en x = 0 (función con esquina)

La derivada clásica NO existe en x = 0

==========

Resultado numérico en x = 0 : -0.0000000000

Derivada lateral izquierda : -1.0

Derivada lateral derecha : 1.0

ANÁLISIS:

La fórmula de 5 puntos usa simetría: los términos negativos y positivos

se cancelan y da exactamente 0.0. Esto parece 'limpio', pero es incorrecto.

La derivada de |x| no existe en x=0 porque los límites laterales son distintos.

CAUSA: La fórmula asume que la función es diferenciable en el punto.

En una esquina (cambio de pendiente abrupto), el supuesto falla.

SOLUCIÓN: Verificar diferenciabilidad antes de aplicar cualquier fórmula.

Usar derivadas laterales si la función tiene esquinas.

Ejercicio 3 (Caso Extra)

Enunciado: En análisis de vibraciones, la velocidad y aceleración de un sistema masa-resorte siguen la ley $x(t) = A\cdot\cos(\omega\cdot t + \varphi)$. Con $A = 0.05$ m, $\omega = 10$ rad/s y $\varphi = \pi/6$, calcular la velocidad y la aceleración en $t = 0.1$ s usando cinco puntos, y también estimar la tercera derivada (jerk) del sistema.

Datos: $x(t) = 0.05\cdot\cos(10t + \pi/6)$, $t_0 = 0.1$, $h = 0.005$

Qué se pide: Calcular $x'(0.1)$, $x''(0.1)$ y $x'''(0.1)$ usando diferencias finitas de cinco puntos adaptadas, y comparar con los valores exactos.

```
def caso_extra(self):
        """
        CASO EXTRA: Sistema masa-resorte x(t) = A·cos(ω·t + ϕ)
        ---------------------------------------------------------------
        Calcular velocidad (x'), aceleración (x'') y jerk (x''') usando
        las fórmulas de 5 puntos para la 1ra, 2da y 3ra derivada.

        Datos:
            x(t) = 0.05·cos(10t + π/6)
            A = 0.05 m, ω = 10 rad/s, ϕ = π/6
            t₀ = 0.1 s
            h = 0.005 s

        Se pide: x'(0.1), x''(0.1), x'''(0.1) y comparar con exactas.
        """
        print("\n" + "="*60)
        print("MÉTODO DE CINCO PUNTOS - CASO EXTRA")
        print("Sistema masa-resorte: x(t) = 0.05·cos(10t + π/6)")
        print("Punto: t₀ = 0.1 s | Paso: h = 0.005 s")
        print("Se calculan: velocidad, aceleración y jerk")
        print("="*60)

        A = 0.05
        omega = 10
        phi = math.pi / 6

        def x(t):
```

```
            return A * math.cos(omega * t + phi)

        # Derivadas exactas
        def x_prima(t):         # velocidad: -A·ω·sin(ω·t + ϕ)
            return -A * omega * math.sin(omega * t + phi)

        def x_doble_prima(t):   # aceleración: -A·ω²·cos(ω·t + ϕ)
            return -A * omega**2 * math.cos(omega * t + phi)

        def x_triple_prima(t):  # jerk: A·ω³·sin(ω·t + ϕ)
            return A * omega**3 * math.sin(omega * t + phi)

        t0 = 0.1
        h = 0.005

        # Calcular con fórmulas de 5 puntos
        vel_aprox   = self.derivada_primera(x, t0, h)
        acel_aprox  = self.derivada_segunda(x, t0, h)
        jerk_aprox  = self.derivada_tercera(x, t0, h)

        vel_exacta  = x_prima(t0)
        acel_exacta = x_doble_prima(t0)
        jerk_exacta = x_triple_prima(t0)

        print(f"\n  VELOCIDAD en t = {t0} s:")
        print(f"    Aproximada  : {vel_aprox:.10f} m/s")
        print(f"    Exacta      : {vel_exacta:.10f} m/s")
        print(f"    Error abs   : {abs(vel_aprox - vel_exacta):.2e}")

        print(f"\n  ACELERACIÓN en t = {t0} s:")
        print(f"    Aproximada  : {acel_aprox:.8f} m/s²")
        print(f"    Exacta      : {acel_exacta:.8f} m/s²")
        print(f"    Error abs   : {abs(acel_aprox - acel_exacta):.2e}")

        print(f"\n  JERK (3ra derivada) en t = {t0} s:")
        print(f"    Aproximada  : {jerk_aprox:.6f} m/s³")
        print(f"    Exacta      : {jerk_exacta:.6f} m/s³")
        print(f"    Error abs   : {abs(jerk_aprox - jerk_exacta):.2e}")

        print("\n  CONCLUSIÓN: El método de 5 puntos permite estimar la 1ra, 2da
y 3ra")
        print("  derivada con alta precisión para funciones analíticas suaves.")
```

Salida:

MÉTODO DE CINCO PUNTOS - CASO EXTRA

Sistema masa-resorte: $x(t) = 0.05 \cdot \cos(10t + \pi/6)$

Punto: $t_0 = 0.1$ s | Paso: $h = 0.005$ s

Se calculan: velocidad, aceleración y jerk

==========

VELOCIDAD en t = 0.1 s:

Aproximada : -0.4994430971 m/s

Exacta : -0.4994432012 m/s

Error abs : 1.04e-07

ACELERACIÓN en t = 0.1 s:

Aproximada : -0.23590013 m/s^2

Exacta : -0.23590015 m/s^2

Error abs : 1.64e-08

JERK (3ra derivada) en t = 0.1 s:

Aproximada : 49.913113 m/s^3

Exacta : 49.944320 m/s^3

Error abs : 3.12e-02

CONCLUSIÓN: El método de 5 puntos permite estimar la 1ra, 2da y 3ra derivada con alta precisión para funciones analíticas suaves.

3.3 EJERCICIOS: MÉTODO DEL TRAPECIO

Ejercicio 1 (Caso Ideal)

Enunciado: Una empresa calcula el trabajo realizado por una fuerza variable F(x) = x·e^(-0.5x) + 2 (en Newtons) sobre un desplazamiento de 0 a 4 metros. Se pide calcular el trabajo total W = $\int_0^4$ F(x) dx usando el método del trapecio compuesto.

Datos: F(x) = x·e^(-0.5x) + 2, a = 0, b = 4, n = 1000

Qué se pide: Calcular la integral numérica y comparar con el valor de referencia obtenido con scipy.

```
def caso_ideal(self):
        """
        CASO IDEAL: W = ∫₀⁴ F(x) dx con F(x) = x·e^(-0.5x) + 2
        ------------------------------------------------------------
```

```
        Contexto: Trabajo realizado por una fuerza variable sobre un
        desplazamiento de 0 a 4 metros.

        Datos:
            F(x) = x·e^(-0.5x) + 2  (Newtons)
            a = 0 m,  b = 4 m
            n = 1000 subintervalos

        Se pide: Calcular el trabajo total W en Joules.
        """
        print("\n" + "="*60)
        print("MÉTODO DEL TRAPECIO - CASO IDEAL")
        print("Función: F(x) = x·e^(-0.5x) + 2 (fuerza en Newtons)")
        print("Intervalo: [0, 4] m | n = 1000 subintervalos")
        print("Contexto: Trabajo realizado por fuerza variable")
        print("="*60)

        def F(x):
            return x * math.exp(-0.5 * x) + 2

        a, b, n = 0, 4, 1000

        aprox = self.integrar(F, a, b, n)
        exacto, _ = integrate.quad(F, a, b)
        error = abs((aprox - exacto) / exacto) * 100

        print(f"\n  Trabajo aproximado (trapecio)  : {aprox:.8f} J")
        print(f"  Valor de referencia (scipy)    : {exacto:.8f} J")
        print(f"  Error relativo                 : {error:.8f}%")
        print("\n  CONCLUSIÓN: Con n=1000, el trapecio converge con error menor
al 0.001%.")
        print("  El método es apropiado para esta función suave y continua.")
```

Salida: MÉTODO DEL TRAPECIO - CASO IDEAL

Función: F(x) = x·e^(-0.5x) + 2 (fuerza en Newtons)

Intervalo: [0, 4] m | n = 1000 subintervalos

Contexto: Trabajo realizado por fuerza variable

==========

Trabajo aproximado (trapecio) : 10.37597509 J

Valor de referencia (scipy) : 10.37597660 J

Error relativo : 0.00001459%

CONCLUSIÓN: Con n=1000, el trapecio converge con error menor al 0.001%.

El método es apropiado para esta función suave y continua.

Ejercicio 2 (Caso con Error)

Enunciado: Se intenta integrar con el trapecio usando muy pocos subintervalos (n = 2) la función f(x) = sin(10x) en [0, π], que oscila rápidamente. El resultado es muy impreciso.

Datos: f(x) = sin(10x), a = 0, b = π, n = 2

Qué se pide: Calcular la integral con n = 2 y con n = 1000, mostrar la diferencia de error y explicar por qué un n bajo falla para funciones oscilantes.\

```
def caso_error(self):
        """
        CASO CON ERROR: ∫₀^π sin(10x) dx con n muy pequeño
        ------------------------------------------------------------
        Sin(10x) oscila 5 veces completas en [0, π]. Con n=2, el trapecio
        no puede capturar estas oscilaciones y el error es enorme.

        Datos:
            f(x) = sin(10x)
            a = 0, b = π
            n = 2, 10, 100, 1000 (análisis comparativo)

        Se pide: Mostrar cómo el error disminuye al aumentar n.
        """
        print("\n" + "="*60)
        print("MÉTODO DEL TRAPECIO - CASO CON ERROR")
        print("Función: f(x) = sin(10x) (función altamente oscilatoria)")
        print("Análisis del error en función del número de subintervalos n")
        print("="*60)

        def f(x):
            return math.sin(10 * x)

        a, b = 0, math.pi
        valor_exacto, _ = integrate.quad(f, a, b)

        print(f"\n  {'n':>6} | {'Aproximación':>15} | {'Error absoluto':>16} |
{'Error rel %':>12}")
        print(f"  {'-'*6}-+-{'-'*15}-+-{'-'*16}-+-{'-'*12}")

        for n in [2, 5, 10, 50, 100, 1000]:
            aprox = self.integrar(f, a, b, n)
            error_abs = abs(aprox - valor_exacto)
            if abs(valor_exacto) > 1e-10:
                error_rel = abs((aprox - valor_exacto) / valor_exacto) * 100
            else:
```

```
                error_rel = error_abs * 100  # Si el exacto es ~0
            print(f"  {n:>6} | {aprox:>15.8f} | {error_abs:>16.8f} |
{error_rel:>12.4f}%")

        print(f"\n  Valor de referencia (scipy): {valor_exacto:.10f}")
        print("\n  CAUSA: Con n pequeño, el trapecio ignora la mayoría de las
oscilaciones.")
        print("  Para sin(10x) en [0,π] existen 5 ciclos completos.")
        print("  REGLA PRÁCTICA: n debe ser >> 1/h_mínimo_de_la_función ≈ 10
ciclos/unidad.")
        print("  SOLUCIÓN: Usar n >= 500 para sin(10x) en [0,π].")
```

Salida: MÉTODO DEL TRAPECIO - CASO CON ERROR

Función: f(x) = sin(10x) (función altamente oscilatoria)

Análisis del error en función del número de subintervalos n

==========

n	Aproximación	Error absoluto	Error rel %
2	0.00000000	0.00000000	0.0000%
5	-0.00000000	0.00000000	0.0000%
10	0.00000000	0.00000000	0.0000%
50	-0.00000000	0.00000000	0.0000%
100	-0.00000000	0.00000000	0.0000%
1000	0.00000000	0.00000000	0.0000%

Valor de referencia (scipy): -0.0000000000

CAUSA: Con n pequeño, el trapecio ignora la mayoría de las oscilaciones.

Para sin(10x) en [0,π] existen 5 ciclos completos.

REGLA PRÁCTICA: n debe ser >> 1/h_mínimo_de_la_función ≈ 10 ciclos/unidad.

SOLUCIÓN: Usar n >= 500 para sin(10x) en [0,π].

Ejercicio 3 (Caso Extra)

Enunciado: En hidrología, el caudal Q(t) de un río (en m³/s) durante 24 horas fue medido experimentalmente en intervalos de 2 horas. Los datos tabulados son: t = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24], Q = [5.0, 6.2, 8.1, 11.3, 14.0, 13.5, 12.0, 10.2, 8.8, 7.4, 6.5, 5.8, 5.1]. Calcular el volumen total de agua que pasó en esas 24 horas.

Datos: Datos tabulados, h = 2 horas

Qué se pide: Calcular $V = \int_0^{24} Q(t)\, dt$ en m³ usando el método del trapecio sobre los datos tabulados. Expresar el resultado en m³ y en litros.

```
def caso_extra(self):
        """
        CASO EXTRA: Volumen de agua en río V = ∫₀²⁴ Q(t) dt con datos tabulados
        ----------------------------------------------------------------
        Se tienen mediciones de caudal Q(t) cada 2 horas durante 24 horas.
        No se conoce la fórmula exacta, solo los datos experimentales.

        Datos (mediciones de campo):
            t (h) : [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]
            Q(m³/s): [5.0, 6.2, 8.1, 11.3, 14.0, 13.5, 12.0, 10.2, 8.8, 7.4, 6.5,
5.8, 5.1]

        Se pide: Calcular el volumen total de agua V en m³ y en litros.
        """
        print("\n" + "="*60)
        print("MÉTODO DEL TRAPECIO - CASO EXTRA")
        print("Datos tabulados: caudal Q(t) de un río durante 24 horas")
        print("Contexto: Hidrología - Cálculo de volumen de escorrentía")
        print("="*60)

        t = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]         # horas
        Q = [5.0, 6.2, 8.1, 11.3, 14.0, 13.5, 12.0, 10.2, 8.8, 7.4, 6.5, 5.8,
5.1]  # m³/s

        print(f"\n  Datos de caudal:")
        print(f"  {'t (h)':>6} | {'Q (m³/s)':>10}")
        print(f"  {'-'*6}-+-{'-'*10}")
        for i in range(len(t)):
            print(f"  {t[i]:>6} | {Q[i]:>10.1f}")

        # Integrar: ∫₀²⁴ Q(t) dt con t en horas → resultado en m³·h/s
        # Convertir a m³: multiplicar por 3600 s/h
        volumen_horas = self.integrar_tabular(t, Q)   # en m³·h/s
        volumen_m3 = volumen_horas * 3600              # convertir h→s

        caudal_promedio = volumen_horas / (t[-1] - t[0])  # m³/s promedio

        print(f"\n  RESULTADOS:")
        print(f"  Integral ∫Q dt (en m³·h/s)         : {volumen_horas:.4f}")
```

```
        print(f"  Volumen total de agua                : {volumen_m3:,.2f} m³")
        print(f"  Volumen total de agua                : {volumen_m3 * 1000:,.0f}
litros")
        print(f"  Volumen total de agua                : {volumen_m3 / 1e6:.6f}
km³")
        print(f"  Caudal promedio en 24 h              : {caudal_promedio:.4f}
m³/s")
```

Salida:

MÉTODO DEL TRAPECIO - CASO EXTRA

Datos tabulados: caudal Q(t) de un río durante 24 horas

Contexto: Hidrología - Cálculo de volumen de escorrentía

==========

Datos de caudal:

t (h)	Q (m³/s)
0	5.0
2	6.2
4	8.1
6	11.3
8	14.0
10	13.5
12	12.0
14	10.2
16	8.8
18	7.4
20	6.5
22	5.8
24	5.1

RESULTADOS:

Integral ∫Q dt (en m³·h/s) : 217.7000

Volumen total de agua: 783,720.00 m^3

Volumen total de agua: 783,720,000 litros

Volumen total de agua: 0.783720 km^3

Caudal promedio en 24 h: 9.0708 m^3/s

3.4 EJERCICIOS: REGLA DE SIMPSON

Ejercicio 1 (Caso Ideal)

Enunciado: Un ingeniero químico necesita calcular la entalpía de reacción integrando la capacidad calorífica $Cp(T) = 28.58 + 3.77\times10^{-3}\cdot T + 0.5\times10^{5}\cdot T^{-2}$ (J/mol·K) entre $T_1 = 300$ K y $T_2 = 1000$ K para el nitrógeno gaseoso.

Datos: $Cp(T) = 28.58 + 3.77e\text{-}3\cdot T + 0.5e5\cdot T^{-2}$, a = 300, b = 1000, n = 100

Qué se pide: Calcular $\Delta H = \int_{300}^{1000} Cp(T)\, dT$ con la regla de Simpson 1/3 y comparar con el valor de referencia.

```
def caso_ideal(self):
        """
        CASO IDEAL: ΔH = ∫₃₀₀¹⁰⁰⁰ Cp(T) dT para nitrógeno gaseoso
        ----------------------------------------------------------------
        Contexto: Ingeniería química - Cálculo de entalpía de calentamiento.
        Cp(T) = 28.58 + 3.77×10⁻³·T + 0.5×10⁵·T⁻² (J/mol·K) para N₂.

        Datos:
            Cp(T) = 28.58 + 3.77e-3·T + 0.5e5·T⁻²
            a = 300 K,  b = 1000 K
            n = 100 subintervalos (par)

        Se pide: ΔH en J/mol y comparar con valor de referencia.
        """
        print("\n" + "="*60)
        print("REGLA DE SIMPSON - CASO IDEAL")
        print("Función: Cp(T) = 28.58 + 3.77e-3·T + 0.5e5/T²  (J/mol·K)")
        print("Intervalo: [300 K, 1000 K] | n = 100")
        print("Contexto: Entalpía de calentamiento del nitrógeno gaseoso")
        print("="*60)

        def Cp(T):
            return 28.58 + 3.77e-3 * T + 0.5e5 / T**2

        a, b, n = 300, 1000, 100

        aprox = self.integrar(Cp, a, b, n)
        exacto, _ = integrate.quad(Cp, a, b)
        error = abs((aprox - exacto) / exacto) * 100
```

```
        print(f"\n  ΔH aproximado (Simpson)       : {aprox:.6f} J/mol")
        print(f"  ΔH de referencia (scipy)      : {exacto:.6f} J/mol")
        print(f"  Error relativo                : {error:.8f}%")
        print("\n  CONCLUSIÓN: Simpson 1/3 integra Cp(T) con error mínimo.")
        print("  Es mucho más preciso que el trapecio para la misma n,")
        print("  gracias a su aproximación parabólica (error O(h⁴) vs O(h²)).")
```

Salida: REGLA DE SIMPSON - CASO IDEAL

Función: $Cp(T) = 28.58 + 3.77e\text{-}3 \cdot T + 0.5e5/T^2$ (J/mol·K)

Intervalo: [300 K, 1000 K] | n = 100

Contexto: Entalpía de calentamiento del nitrógeno gaseoso

==========

ΔH aproximado (Simpson) : 21838.016673 J/mol

ΔH de referencia (scipy) : 21838.016667 J/mol

Error relativo : 0.00000003%

CONCLUSIÓN: Simpson 1/3 integra Cp(T) con error mínimo.

Es mucho más preciso que el trapecio para la misma n,

gracias a su aproximación parabólica (error $O(h^4)$ vs $O(h^2)$).

Ejercicio 2 (Caso con Error)

Enunciado: Se intenta aplicar Simpson 1/3 con n = 3 (número impar de subintervalos), lo cual viola el requisito del método. El programa debe detectar el error y manejarlo correctamente.

Datos: $f(x) = x^2$, a = 0, b = 3, n = 3 (impar, inválido para Simpson 1/3)

Qué se pide: Mostrar qué ocurre si se intenta aplicar Simpson con n impar, manejar el error con una excepción o advertencia, y recalcular con n = 4.

```
def caso_error(self):
        """
        CASO CON ERROR: Intentar Simpson con n = 3 (impar) sobre f(x) = x²
        ------------------------------------------------------------
        La regla de Simpson 1/3 requiere n par. Si se pasa n impar,
        el algoritmo debe detectarlo y lanzar un error claro.

        Datos:
            f(x) = x²
            a = 0, b = 3, n = 3 (impar → INVÁLIDO)
```

```
            Valor exacto: ∫₀³ x² dx = 27/3 = 9

        Se pide: Capturar el error, explicar la causa y resolver con n = 4.
        """
        print("\n" + "="*60)
        print("REGLA DE SIMPSON - CASO CON ERROR")
        print("Función: f(x) = x²  |  n = 3 (impar → inválido para Simpson 1/3)")
        print("="*60)

        def f(x):
            return x**2

        a, b = 0, 3
        valor_exacto = 9.0  # ∫₀³ x² dx = x³/3 |₀³ = 9

        # --- Intento con n impar ---
        print("\n  Intentando con n = 3 (impar)...")
        try:
            resultado = self.integrar(f, a, b, n=3)
            print(f"  Resultado inesperado: {resultado}")
        except ValueError as e:
            print(f"  EXCEPCIÓN CAPTURADA:\n  {e}")
            print("\n  CAUSA: El patrón 1,4,2,4,...,4,1 de Simpson requiere que")
            print("  los índices impares (coeficiente 4) sean correctamente
alternados.")
            print("  Con n impar, el esquema de coeficientes no cierra
correctamente.")

            # --- Recuperación automática ---
            n_corregido = 4
            print(f"\n  Recalculando con n = {n_corregido} (par)...")
            resultado = self.integrar(f, a, b, n=n_corregido)
            error = abs((resultado - valor_exacto) / valor_exacto) * 100

            print(f"  Resultado con n = {n_corregido}  : {resultado:.10f}")
            print(f"  Valor exacto        : {valor_exacto:.10f}")
            print(f"  Error relativo      : {error:.2e}%")
            print("\n  NOTA: Simpson integra EXACTAMENTE polinomios de grado ≤
3.")
            print("  x² es de grado 2, por eso el error es prácticamente 0.")
```

Salida: REGLA DE SIMPSON - CASO CON ERROR

Función: f(x) = x² | n = 3 (impar → inválido para Simpson 1/3)

==========

Intentando con n = 3 (impar)...

EXCEPCIÓN CAPTURADA:

ERROR: La Regla de Simpson 1/3 requiere número PAR de subintervalos. Se recibió n = 3. Use n = 4 o n = 2.

CAUSA: El patrón 1,4,2,4,...,4,1 de Simpson requiere que

los índices impares (coeficiente 4) sean correctamente alternados.

Con n impar, el esquema de coeficientes no cierra correctamente.

Recalculando con n = 4 (par)...

Resultado con n = 4 : 9.0000000000

Valor exacto : 9.0000000000

Error relativo : 0.00e+00%

NOTA: Simpson integra EXACTAMENTE polinomios de grado ≤ 3.

x^2 es de grado 2, por eso el error es prácticamente 0.

Ejercicio 3 (Caso Extra)

Enunciado: En electromagnetismo, la energía almacenada en un campo magnético a lo largo de un solenoide no uniforme se modela con $U(x) = \mu_0 \cdot (N/L)^2 \cdot A \cdot B(x)^2$, donde $B(x) = B_0 \cdot (1 + 0.2 \cdot \sin(\pi x/L))$. Con $B_0 = 0.5$ T, $L = 2$ m, $A = 0.01$ m², $\mu_0 = 4\pi \times 10^{-7}$, $N/L = 500$ vueltas/m, calcular la energía total $U = \int_0^2 U(x)\,dx$ y también estimar la energía en el primer y segundo metro por separado.

Datos: Ver enunciado, n = 100

Qué se pide: Calcular la energía total y las dos subintegrales con Simpson, mostrando resultados en Joules.

```
def caso_extra(self):
        """
        CASO EXTRA: Energía magnética en solenoide no uniforme
        ------------------------------------------------------------------
        Contexto: Electromagnetismo - Energía almacenada en solenoide con
        densidad de campo variable B(x) = B₀·(1 + 0.2·sin(πx/L)).

        Parámetros:
            B₀ = 0.5 T,  L = 2 m,  A = 0.01 m²
            μ₀ = 4π×10⁻⁷ H/m,  N/L = 500 vueltas/m

        Se pide:
            1. Energía total U en [0, 2] m
```

```
            2. Energía en primer metro [0, 1]
            3. Energía en segundo metro [1, 2]
            4. Verificar que las dos mitades suman la total
        """
        print("\n" + "="*60)
        print("REGLA DE SIMPSON - CASO EXTRA")
        print("Energía magnética en solenoide no uniforme")
        print("B(x) = B₀·(1 + 0.2·sin(πx/L))")
        print("Parámetros: B₀=0.5 T, L=2 m, A=0.01 m², N/L=500 vueltas/m")
        print("="*60)

        mu0 = 4 * math.pi * 1e-7   # Permeabilidad del vacío (H/m)
        B0 = 0.5                    # Campo magnético base (T)
        L = 2.0                     # Longitud del solenoide (m)
        A = 0.01                    # Área de la sección transversal (m²)
        NL = 500                    # Densidad de vueltas (N/L vueltas/m)

        def densidad_energia(x):
            """
            Densidad de energía magnética lineal:
            u(x) = (μ₀/2)·(N/L)²·A·B(x)²
            Donde B(x) = B₀·(1 + 0.2·sin(πx/L))
            """
            B = B0 * (1 + 0.2 * math.sin(math.pi * x / L))
            return (mu0 / 2) * NL**2 * A * B**2

        n = 100  # Par, requerido por Simpson

        energia_total      = self.integrar(densidad_energia, 0, 2, n)
        energia_primer_m   = self.integrar(densidad_energia, 0, 1, n)
        energia_segundo_m  = self.integrar(densidad_energia, 1, 2, n)
        suma_mitades       = energia_primer_m + energia_segundo_m
        discrepancia       = abs(energia_total - suma_mitades)

        print(f"\n  Energía total en [0, 2] m         : {energia_total:.10f} J")
        print(f"  Energía en primer metro [0, 1]    : {energia_primer_m:.10f} J")
        print(f"  Energía en segundo metro [1, 2]   : {energia_segundo_m:.10f}
J")
        print(f"  Suma de las dos mitades           : {suma_mitades:.10f} J")
        print(f"  Discrepancia (debe ser ~0)        : {discrepancia:.2e} J")
        print(f"  Verificación de consistencia      : {'PASSED ✓' if
discrepancia < 1e-12 else 'FAILED ✗'}")

        porcentaje_1m = energia_primer_m / energia_total * 100
        porcentaje_2m = energia_segundo_m / energia_total * 100
        print(f"\n  Distribución de energía:")
        print(f"    Primer metro  : {porcentaje_1m:.2f}% del total")
        print(f"    Segundo metro : {porcentaje_2m:.2f}% del total")
```

Salida: REGLA DE SIMPSON - CASO EXTRA

Energía magnética en solenoide no uniforme

$B(x) = B_0 \cdot (1 + 0.2 \cdot \sin(\pi x/L))$

Parámetros: B_0=0.5 T, L=2 m, A=0.01 m², N/L=500 vueltas/m

==========

Energía total en [0, 2] m : 0.0010011061 J

Energía en primer metro [0, 1] : 0.0005005531 J

Energía en segundo metro [1, 2] : 0.0005005531 J

Suma de las dos mitades : 0.0010011061 J

Discrepancia (debe ser ~0) : 1.01e-12 J

Verificación de consistencia : FAILED ✗

Distribución de energía:

Primer metro : 50.00% del total

Segundo metro : 50.00% del total

3.5 EJERCICIOS: CUADRATURA GAUSSIANA

Ejercicio 1 (Caso Ideal)

Enunciado: En mecánica orbital, la longitud de una trayectoria elíptica de un satélite entre dos ángulos $\theta_1 = 0$ y $\theta_2 = \pi$ se aproxima mediante la integral de arco. Calcular $\int_0^\pi \sqrt{a^2 \cdot \sin^2(\theta) + b^2 \cdot \cos^2(\theta)}\, d\theta$ con a = 7000 km (semieje mayor) y b = 6800 km (semieje menor) usando cuadratura gaussiana con 5 puntos.

Datos: $f(\theta) = \sqrt{7000^2 \cdot \sin^2(\theta) + 6800^2 \cdot \cos^2(\theta)}$, a = 0, b = π, n = 5

Qué se pide: Calcular la longitud del arco y comparar con los resultados de 2, 5 y 10 puntos gaussianos para demostrar la convergencia rápida.

```
def caso_ideal(self):
        """
        CASO IDEAL: Longitud de arco orbital de satélite elíptico
        ------------------------------------------------------------
        La longitud del arco de una elipse en [0, π] es:
            L = ∫₀^π √(a²·sin²(θ) + b²·cos²(θ)) dθ

        con a = 7000 km (semieje mayor) y b = 6800 km (semieje menor).
        Se compara la convergencia con 2, 5, 10 y 20 puntos gaussianos.
```

```
        Datos:
            f(θ) = √(7000²·sin²(θ) + 6800²·cos²(θ))
            a_integ = 0, b_integ = π

        Se pide: Calcular la longitud y mostrar convergencia.
        """
        print("\n" + "="*60)
        print("CUADRATURA GAUSSIANA - CASO IDEAL")
        print("Longitud de arco orbital: ∫₀^π √(a²sin²θ + b²cos²θ) dθ")
        print("Semiejes: a = 7000 km, b = 6800 km")
        print("="*60)

        a_orb = 7000.0   # km
        b_orb = 6800.0   # km

        def f(theta):
            return math.sqrt(a_orb**2 * math.sin(theta)**2 + b_orb**2 *
math.cos(theta)**2)

        referencia, _ = integrate.quad(f, 0, math.pi)

        print(f"\n  {'n':>4} | {'Aproximación (km)':>20} | {'Error relativo
%':>18}")
        print(f"  {'-'*4}-+-{'-'*20}-+-{'-'*18}")

        for n in [2, 3, 5, 10, 20]:
            aprox = self.integrar(f, 0, math.pi, n)
            error = abs((aprox - referencia) / referencia) * 100
            print(f"  {n:>4} | {aprox:>20.6f} | {error:>18.8f}%")

        print(f"\n  Valor de referencia (scipy): {referencia:.6f} km")
        print("\n  CONCLUSIÓN: Con solo 5 puntos gaussianos se logra un error <
0.0001%.")
        print("  La cuadratura gaussiana converge exponencialmente rápido para
funciones suaves.")
```

Salida: CUADRATURA GAUSSIANA - CASO IDEAL

Longitud de arco orbital: ∫0^π √(a²sin²θ + b²cos²θ) dθ

Semiejes: a = 7000 km, b = 6800 km

==========

n	Aproximación (km)	Error relativo %
2	21603.548884	0.34402740%

3	21684.584473	0.02978528%
5	21678.120735	0.00003159%
10	21678.127583	0.00000000%
20	21678.127583	0.00000000%

Valor de referencia (scipy): 21678.127583 km

CONCLUSIÓN: Con solo 5 puntos gaussianos se logra un error < 0.0001%.

La cuadratura gaussiana converge exponencialmente rápido para funciones suaves.

Ejercicio 2 (Caso con Error)

Enunciado: Se intenta usar cuadratura gaussiana con solo 2 puntos para integrar f(x) = cos(20x) en [0, π], una función altamente oscilatoria. El resultado difiere enormemente del valor exacto (que es 0), demostrando la limitación del método ante funciones oscilatorias con pocos puntos.

Datos: f(x) = cos(20x), a = 0, b = π, n = 2, 5, 10, 20

Qué se pide: Mostrar el resultado con 2, 5, 10 y 20 puntos y comparar con el valor exacto 0, analizando cuántos puntos se necesitan para converger.

```
def caso_error(self):
        """
        CASO CON ERROR: ∫₀^π cos(20x) dx (función muy oscilatoria)
        ------------------------------------------------------------
        El valor exacto de ∫₀^π cos(20x) dx = sin(20π)/20 ≈ 0.
        Con pocos puntos gaussianos, el resultado es incorrecto porque
        los nodos no pueden muestrear adecuadamente las oscilaciones.

        Datos:
            f(x) = cos(20x)
            a = 0, b = π
            Valor exacto ≈ 0

        Se pide: Mostrar la convergencia con n = 2, 5, 10, 20, 50.
        """
        print("\n" + "="*60)
        print("CUADRATURA GAUSSIANA - CASO CON ERROR")
        print("Función: f(x) = cos(20x) en [0, π] (altamente oscilatoria)")
        print("Valor exacto = sin(20π)/20 ≈ 0")
        print("="*60)

        def f(x):
            return math.cos(20 * x)
```

```
        valor_exacto = math.sin(20 * math.pi) / 20  # ≈ 0 (exactamente 0)

        print(f"\n  {'n':>4} | {'Aproximación':>16} | {'Error abs':>14} |
{'Convergió':>10}")
        print(f"  {'-'*4}-+-{'-'*16}-+-{'-'*14}-+-{'-'*10}")

        for n in [2, 5, 10, 20, 50, 100]:
            aprox = self.integrar(f, 0, math.pi, n)
            error_abs = abs(aprox - valor_exacto)
            convergio = "SI" if error_abs < 1e-4 else "NO"
            print(f"  {n:>4} | {aprox:>16.8f} | {error_abs:>14.8f} |
{convergio:>10}")

        print(f"\n  Valor exacto: {valor_exacto:.15f}")
        print("\n  CAUSA: cos(20x) completa 10 ciclos en [0,π]. La cuadratura
gaussiana")
        print("  con n puntos integra exactamente polinomios de grado 2n-1, pero
NO")
        print("  funciones trigonométricas oscilatorias con pocos puntos.")
        print("  SOLUCIÓN: Usar n >= 50, o dividir [0,π] en subintervalos
pequeños")
        print("  y aplicar cuadratura gaussiana en cada uno (cuadratura
adaptativa).")
```

Salida: CUADRATURA GAUSSIANA - CASO CON ERROR

Función: f(x) = cos(20x) en [0, π] (altamente oscilatoria)

Valor exacto = sin(20π)/20 ≈ 0

==========

n	Aproximación	Error abs	Convergió
2	2.37926196	2.37926196	NO
5	-0.36959600	0.36959600	NO
10	-0.23157767	0.23157767	NO
20	-0.00716678	0.00716678	NO
50	0.00000000	0.00000000	SI
100	-0.00000000	0.00000000	SI

Valor exacto: -0.000000000000000

CAUSA: cos(20x) completa 10 ciclos en [0,π]. La cuadratura gaussiana con n puntos integra exactamente polinomios de grado 2n-1, pero NO funciones trigonométricas oscilatorias con pocos puntos.

SOLUCIÓN: Usar n >= 50, o dividir [0,π] en subintervalos pequeños y aplicar cuadratura gaussiana en cada uno (cuadratura adaptativa).

Ejercicio 3 (Caso Extra)

Enunciado: En probabilidad, la función de distribución normal estándar acumulada es $\Phi(z) = (1/\sqrt{(2\pi)}) \cdot \int_0^z e^{(-t^2/2)}\, dt$. Calcular $P(Z \le 1.96)$, $P(Z \le 1.0)$ y $P(Z \le 2.576)$ usando cuadratura gaussiana con 10 puntos, y compararlos con los valores tabulados de la distribución normal.

Datos: $f(t) = (1/\sqrt{(2\pi)})\cdot e^{(-t^2/2)}$, límites 0 a z, n = 10

Qué se pide: Estimar las tres probabilidades y comparar con scipy.stats.norm.cdf().

```
def caso_extra(self):
        """
        CASO EXTRA: Distribución Normal Estándar acumulada Φ(z)
        ----------------------------------------------------------------
        Calcular P(Z ≤ z) = 0.5 + (1/√(2π)) · ∫₀ᶻ e^(-t²/2) dt
        para z = 1.0, 1.96 y 2.576 usando cuadratura gaussiana con 10 puntos.

        Comparar con scipy.stats.norm.cdf() (valor de referencia).

        Datos:
            f(t) = (1/√(2π)) · e^(-t²/2)
            Integrar de 0 a z
            n = 10 puntos gaussianos

        Se pide: P(Z ≤ 1.0), P(Z ≤ 1.96), P(Z ≤ 2.576)
        """
        print("\n" + "="*60)
        print("CUADRATURA GAUSSIANA - CASO EXTRA")
        print("Distribución Normal Estándar Acumulada Φ(z)")
        print("Φ(z) = 0.5 + (1/√(2π))·∫₀ᶻ e^(-t²/2) dt")
        print("="*60)

        def integrando_normal(t):
            return (1.0 / math.sqrt(2 * math.pi)) * math.exp(-t**2 / 2)

        valores_z = {
            1.0:   "Intervalo ±1σ cubre el 68.27% de la distribución",
            1.96:  "Intervalo de confianza 95% (valor z crítico)",
            2.576: "Intervalo de confianza 99% (valor z crítico)"
```

```
        }

        print(f"\n  {'z':>7} | {'Φ(z) Gauss':>14} | {'Φ(z) scipy':>14} | {'Error
abs':>12} | Interpretación")
        print(f"  {'-'*7}-+-{'-'*14}-+-{'-'*14}-+-{'-'*12}-+-" + "-"*40)

        n = 10

        for z, interpretacion in valores_z.items():
            integral = self.integrar(integrando_normal, 0, z, n)
            phi_gauss = 0.5 + integral
            phi_scipy = stats.norm.cdf(z)
            error = abs(phi_gauss - phi_scipy)
            print(f"  {z:>7.3f} | {phi_gauss:>14.8f} | {phi_scipy:>14.8f} |
{error:>12.2e} | {interpretacion}")

        print("\n  CONCLUSIÓN: Con solo 10 puntos gaussianos se obtienen 8 cifras
decimales")
        print("  correctas para la distribución normal. Esto requeriría miles de
puntos")
        print("  con el método del trapecio o Simpson para igual precisión.")
        print("  La razón: e^(-t²/2) es extremadamente suave → convergencia
exponencial.")
```

Salida: CUADRATURA GAUSSIANA - CASO EXTRA

Distribución Normal Estándar Acumulada Φ(z)

Φ(z) = 0.5 + (1/√(2π))·∫₀ᶻ e^(-t²/2) dt

==========

Z	Φ(z) Gauss	Φ(z) scipy	Error abs	Interpretación
1.000	0.84134475	0.84134475	0.00e+00	Intervalo ±1σ cubre el 68.27% de la distribución
1.960	0.97500210	0.97500210	1.11e-16	Intervalo de confianza 95% (valor z crítico)
2.576	0.99500247	0.99500247	2.44e-14	Intervalo de confianza 99% (valor z crítico)

CONCLUSIÓN: Con solo 10 puntos gaussianos se obtienen 8 cifras decimales

correctas para la distribución normal. Esto requeriría miles de puntos

con el método del trapecio o Simpson para igual precisión.

La razón: e^(-t²/2) es extremadamente suave → convergencia exponencial.

Commit a GitHub

```
PS C:\Escuela\4to\Métodos_numéricos> git add .
PS C:\Escuela\4to\Métodos_numéricos> git commit -m "Codigo del problemario"
[main eba7b61] Codigo del problemario
 2 files changed, 1042 insertions(+)
 create mode 100644 tema 2/~$metodosNoMacros.xlsx
 create mode 100644 tema 4/Problemario.py
PS C:\Escuela\4to\Métodos_numéricos> git push
Enter passphrase for key '/c/Users/emanu/.ssh/id_ed25519':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 11.14 KiB | 543.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:DeificShoe/Metodos_numericos.git
   cc64a6e..eba7b61  main -> main
PS C:\Escuela\4to\Métodos_numéricos>
```